

MovieWk2

AUTHOR

Andre Thomson

Approach

For this assignment, I asked six people to rate six animated movies. I used participant numbers instead of names. The rating scale was:

- 1 = Did not like it
- 2 = It was okay
- 3 = I liked it
- 4 = I really liked it
- 5 = Loved it

Each person rated only the movies they had seen. If they had not seen a movie, they could leave it blank. I saved the ratings in SQLite and used R to look at the results.

Database design

I split the data into three tables. The `users` table has the participant numbers, the `movies` table has the movie titles, and the `ratings` table has the scores.

The `ratings` table uses `user_id` and `movie_id` together as its primary key. This keeps a person from rating the same movie twice. I also used foreign keys to connect the tables and a `CHECK` rule to keep the ratings between 1 and 5.

The `movie_ratings.sql` file has all the code used to create the tables, add the ratings, and check the results.

Tools used

I used Google Forms to collect the ratings. I used SQLite for the database and ran the SQL in SQLiteOnline. I used Posit Cloud for the R work and to render this report. SQLite uses a local database file, so there is no database password in my code.

I also used Visual Studio Code with GitHub Copilot to help fix errors in the SQL and R code. I checked the suggestions against the final output.

Load the database

The SQLite database must be saved as `movie_ratings.sqlite` in the same folder as this Quarto file.

```

library(DBI)
library(RSQLite)
library(dplyr)
library(tidyr)
library(ggplot2)
library(knitr)

db_file <- "movie_ratings.sqlite"

if (!file.exists(db_file)) {
  stop("movie_ratings.sqlite is missing. Run movie_ratings.sql first.")
}

con <- dbConnect(SQLite(), db_file)

```

Load the SQL data into R

I joined the three tables so I could see the participant number, movie title, and rating in one dataframe. Each row is one completed rating.

```

ratings <- dbGetQuery(
  con,
  "SELECT
    u.user_id,
    u.participant_id,
    m.movie_id,
    m.title AS movie_title,
    r.rating
  FROM ratings AS r
  INNER JOIN users AS u
    ON r.user_id = u.user_id
  INNER JOIN movies AS m
    ON r.movie_id = m.movie_id
  ORDER BY u.user_id, m.movie_id"
) |>
  as_tibble()

users <- dbReadTable(con, "users") |> as_tibble()
movies <- dbReadTable(con, "movies") |> as_tibble()

```

I checked the data before calculating the results.

```

if (nrow(users) < 5) {
  stop("The assignment requires at least five participants.")
}

if (any(!ratings$rating %in% 1:5)) {
  stop("A rating is outside the 1-5 scale.")
}

```

```

}

if (anyDuplicated(ratings[c("user_id", "movie_id")]) > 0) {
  stop("A participant rated the same movie more than once.")
}

data_check <- tibble(
  check = c("Participants", "Movies", "Completed ratings"),
  result = c(nrow(users), nrow(movies), nrow(ratings))
)

kable(data_check, col.names = c("Check", "Result"))

```

Check	Result
Participants	6
Movies	6
Completed ratings	36

Collected ratings

```

ratings |>
  select(participant_id, movie_title, rating) |>
  kable(
    col.names = c("Participant", "Movie", "Rating"),
    align = c("l", "l", "r")
  )

```

Participant	Movie	Rating
Participant 1	Puss in Boots: The Last Wish	5
Participant 1	The Wild Robot	5
Participant 1	Inside Out 2	5
Participant 1	Moana 2	5
Participant 1	Dog Man	3
Participant 1	The Super Mario Bros. Movie	4
Participant 2	Puss in Boots: The Last Wish	5
Participant 2	The Wild Robot	5
Participant 2	Inside Out 2	3
Participant 2	Moana 2	5
Participant 2	Dog Man	2
Participant 2	The Super Mario Bros. Movie	4

Participant	Movie	Rating
Participant 3	Puss in Boots: The Last Wish	3
Participant 3	The Wild Robot	2
Participant 3	Inside Out 2	1
Participant 3	Moana 2	2
Participant 3	Dog Man	5
Participant 3	The Super Mario Bros. Movie	5
Participant 4	Puss in Boots: The Last Wish	3
Participant 4	The Wild Robot	5
Participant 4	Inside Out 2	2
Participant 4	Moana 2	3
Participant 4	Dog Man	4
Participant 4	The Super Mario Bros. Movie	5
Participant 5	Puss in Boots: The Last Wish	5
Participant 5	The Wild Robot	4
Participant 5	Inside Out 2	1
Participant 5	Moana 2	1
Participant 5	Dog Man	1
Participant 5	The Super Mario Bros. Movie	5
Participant 6	Puss in Boots: The Last Wish	5
Participant 6	The Wild Robot	5
Participant 6	Inside Out 2	3
Participant 6	Moana 2	3
Participant 6	Dog Man	2
Participant 6	The Super Mario Bros. Movie	5

Missing ratings

I did not use zero when somebody had not seen a movie. Zero is not part of the scale, and using it would make the average too low. A blank answer is stored by leaving out that rating row.

I checked every person against every movie. If a rating was not there, R showed it as **NA**.

```
all_combinations <- crossing(
  users |> select(user_id, participant_id),
  movies |> select(movie_id, movie_title = title)
)
```

```
complete_ratings <- all_combinations |>
  left_join(
    ratings |> select(user_id, movie_id, rating),
    by = c("user_id", "movie_id")
  )

missing_ratings <- complete_ratings |>
  filter(is.na(rating))

if (nrow(missing_ratings) == 0) {
  cat("There are no missing ratings in this dataset.")
} else {
  missing_ratings |>
    select(participant_id, movie_title) |>
    kable(col.names = c("Participant", "Movie left blank"))
}
```

There are no missing ratings in this dataset.

No one left a movie blank in this set of responses. The code still looks for missing ratings and would keep them as **NA** rather than zero.

Ratings by participant

```
participant_summary <- complete_ratings |>
  group_by(participant_id) |>
  summarise(
    movies Rated = sum(!is.na(rating)),
    movies_left_blank = sum(is.na(rating)),
    .groups = "drop"
  )

kable(
  participant_summary,
  col.names = c("Participant", "Movies rated", "Movies left blank")
)
```

Participant	Movies rated	Movies left blank
Participant 1	6	0
Participant 2	6	0
Participant 3	6	0
Participant 4	6	0
Participant 5	6	0
Participant 6	6	0

Average rating by movie

```
movie_summary <- complete_ratings |>
  group_by(movie_title) |>
  summarise(
    number_of_ratings = sum(!is.na(rating)),
    average_rating = mean(rating, na.rm = TRUE),
    .groups = "drop"
  ) |>
  mutate(
    average_rating = if_else(
      is.nan(average_rating),
      NA_real_,
      average_rating
    )
  ) |>
  arrange(desc(average_rating), movie_title)

movie_summary |>
  mutate(average_rating = round(average_rating, 2)) |>
  kable(
    col.names = c("Movie", "Number of ratings", "Average rating"),
    align = c("l", "r", "r")
  )
```

Movie	Number of ratings	Average rating
The Super Mario Bros. Movie	6	4.67
Puss in Boots: The Last Wish	6	4.33
The Wild Robot	6	4.33
Moana 2	6	3.17
Dog Man	6	2.83
Inside Out 2	6	2.50

I showed the number of ratings with each average so it is clear how many answers were used.

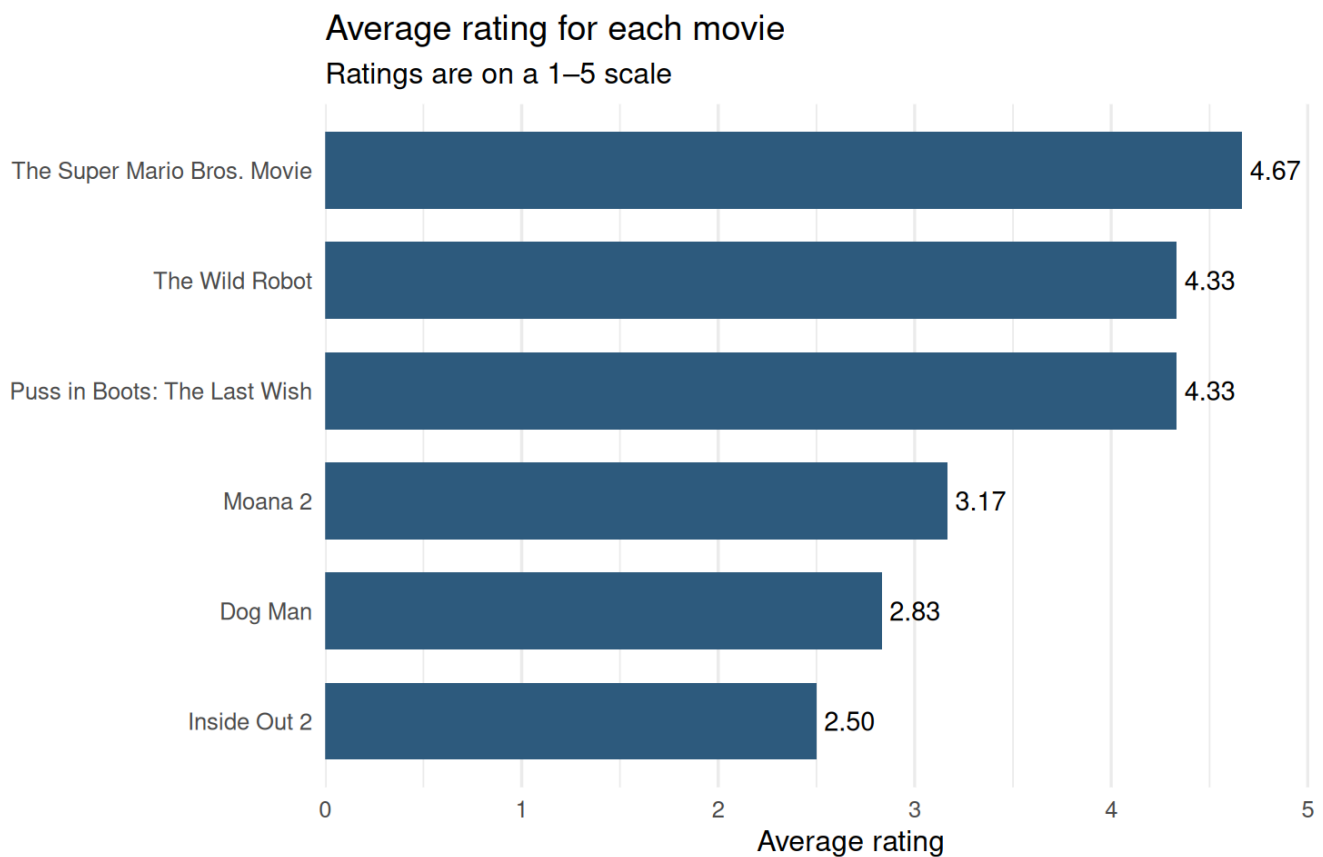
Chart

```
ggplot(
  movie_summary,
  aes(x = reorder(movie_title, average_rating), y = average_rating)
) +
  geom_col(fill = "#315A7D", width = 0.7, na.rm = TRUE) +
  geom_text(
    aes(label = sprintf("%.2f", average_rating)),
    hjust = -0.15,
```

```

    size = 3.8,
    na.rm = TRUE
  ) +
  coord_flip() +
  scale_y_continuous(
    limits = c(0, 5.35),
    breaks = 0:5,
    expand = expansion(mult = c(0, 0))
  ) +
  labs(
    title = "Average rating for each movie",
    subtitle = "Ratings are on a 1–5 scale",
    x = NULL,
    y = "Average rating"
  ) +
  theme_minimal(base_size = 12) +
  theme(panel.grid.major.y = element_blank())

```



Findings

The Super Mario Bros. Movie came out on top for this group, while Inside Out 2 had the lowest average. Everyone answered all six movie questions, so the averages are based on the same group of people. I did not have to remove or fill in any blank ratings for this set of responses.

This was only a group of six people, so the results apply to this group and not to everyone who watches these movies.

Conclusion

This assignment showed me how SQL and R can work together. I created the tables in SQLite, joined them by their IDs, and loaded the results into R. I then checked for blanks and calculated the average rating for each movie.

Sources and references

- [SQLite CREATE TABLE documentation](#) for primary keys, foreign keys, and rating checks.
- [DBI introduction](#) for connecting R to the SQLite database.
- [SQLBolt lessons](#) for a tutorial review of joins and queries.
- Wickham, Çetinkaya-Rundel, and Grolemund, [R for Data Science \(2e\), Chapter 21: Databases](#), for the R/database workflow.
- [Quarto's RStudio tutorial](#) for the report format and rendering steps.
- Mine Çetinkaya-Rundel, [Get started with Quarto](#), a video overview of Quarto.